NAMEI_EXT: A `sched_ext`-Style Extension Point for VFS Name Resolution

Anonymous

摘要

Modern agent, build, and service systems increasingly construct task-specific filesystem views around ordinary tools. The risk is binding a pathname to the wrong lower object, which can corrupt an agent workspace, reuse stale build state, or expose an unintended service configuration. The root cause is a boundary mismatch in which workload state changes pathname-to-object selection during name resolution, while permissions, data operations, writes, persistence, and consistency should remain lower-filesystem responsibilities. Existing mechanisms either materialize a namespace before lookup, mediate access through security hooks, or cross into broader FUSE, stackable, or custom filesystem ownership. We argue that the missing narrow boundary is policy at VFS name resolution, where pathname-to-object selection and directory visibility are decided. Realizing this boundary is hard because policy must run at lookup and readdir, return bounded kernel-validated actions, and remain comparable under the same correctness condition against FUSE and broader filesystem boundaries. NAMEI_EXT implements this ownership split as a `sched_ext`-style VFS extension point. One `cgroup/namei_ext` decision function lets eBPF choose pass, redirect, hide, and `SELECT_TARGET` actions over kernel-registered lower paths while the kernel retains filesystem ownership. Five source-derived RQ1 workflows pass reviewed KVM oracles. In the headline Agent case, two concurrent process groups select different existing Click workspace roots at one pathname; a released task observes 39/40 tests on the base view and 40/40 on the completed view, and switch/rollback changes both object identity and task outcome. Supporting workflows cover sandboxed application sharing, Bazel action views, the already-materialized payload-view subset of Kubernetes `AtomicWriter`, and toolchain environments. On the matched Agent lifecycle, median latency is 5.51 μs versus 62.64 μs for FUSE. On cache-hot FxMark, `SELECT` delivers 1.052–1.088× cached-FUSE throughput across three path-walk tests, while a separate 60-boot experiment meets its predeclared patched-unattached fast-path criterion. On corrected FxMark directory enumeration, `SELECT` delivers 2.20–3.66× FUSE throughput in five of six cells; at four shared-directory workers, the two are indistinguishable. Finally, a matched Wrapfs-derived experiment passes 37/37 workspace oracles and 21/21 fault cells in each of three boots while demonstrating the broader filesystem-method surface owned by the stackable implementation.

1 Introduction

Modern systems increasingly construct task-specific filesystem views around ordinary tools. Agents branch, fork, checkpoint, hide side effects, and run build or test commands over repository workspaces [45, 25, 50]. Environment systems validate repositories against generated Docker state, cache epochs, and verified local artifacts [12, 11, 13, 33], while services consume projected configuration, secrets, certificates, or fixtures as operational state changes [18, 19, 17].

However, these views make pathname binding a correctness boundary. Binding a name to the wrong lower object can validate the wrong revision, leak a write into a parent workspace, feed stale cache input into a build, or expose the wrong service configuration. In many of these cases, the needed behavior is not a new file format, storage service, or custom file operation. The needed behavior chooses which existing object a name denotes, or whether that object is visible.

The root cause is a boundary mismatch in today's filesystem extension points. Workload state changes the right pathname-to-object relation at lookup or directory-enumeration time, but ordinary file data, writes, permissions, page-cache behavior, persistence, and consistency should remain with the lower filesystem. A mechanism that acts before lookup must materialize or update a view, while a mechanism that owns lookup as a filesystem service usually owns more filesystem behavior than this subproblem needs.

Existing mechanisms bracket this lookup-time selection problem. Namespace mechanisms such as bind mounts, OverlayFS, projected volumes, and copied trees construct or materialize a namespace before the application uses it [22, 42, 18]. eBPF LSM hooks can attach verified access-control policy to security decisions [41], but they are not an object-selection boundary. FUSE, stackable filesystems, custom filesystems, and metadata services can compute richer views, but they cross into a broader filesystem boundary [37, 51, 24, 53, 30, 29]. They may be the right mechanism when the oracle needs synthetic contents, custom metadata, data-path mediation, or runtime orchestration, but they are broader than lookup-time name-resolution policy.

We argue that the missing narrow systems boundary is policy at VFS name resolution, between access-control hooks and filesystem-service ownership. We call the recurring subproblem a state-dependent path view, a workload-state change that chooses which existing lower object a pathname denotes or whether that object is visible. VFS name resolution is the point where pathname-to-object selection and directory visibility are decided. The insight is to separate that policy from filesystem ownership by letting a bounded policy choose path-view actions at lookup and readdir time, then returning to ordinary VFS and lower-filesystem semantics.

Making that separation practical creates three technical challenges. First, the policy must run at the lookup and directory-enumeration operations that decide the visible object, not in a separate namespace-construction phase. Second, the policy output must be bounded and kernel-validated so the lower filesystem keeps permissions, writes, data, page cache, persistence, and consistency. Third, the same correctness condition must be usable to compare this boundary with a feature-equivalent FUSE policy service and with the broader ownership required by custom or stackable filesystems.

We present NAMEI_EXT as a `sched_ext`-style extension point for the VFS. eBPF chooses name-resolution policy, while the kernel keeps the VFS mechanisms that own objects and execute filesystem semantics [36]. One eBPF decision function receives lookup and directory-enumeration events, returns bounded pass, redirect, hide, or `SELECT_TARGET` actions over kernel-registered lower paths, and relies on kernel validation before ordinary VFS execution resumes. The policy does not implement file operations, allocate dentries or inodes, run a filesystem daemon, or materialize a namespace tree. Together, these choices answer the three challenges. The hook is placed at name resolution, the action space is bounded and validated, and the mechanism preserves the same lower objects used by the FUSE rows and referenced in the custom/stackable-filesystem boundary account.

We evaluate whether this ownership split is sufficient for source-derived path-view policies, cheaper than feature-equivalent FUSE placement, and more contained than broader filesystem ownership. RQ1 asks whether KVM workloads pass the same correctness conditions while preserving lower-filesystem semantics. Reviewed formal KVM runs pass fixed oracles for Agent workspaces, application-sharing grants, concurrent Bazel action views, the already-materialized payload-view subset of Kubernetes `AtomicWriter`, and toolchain environment selection. The headline Agent result includes three fresh boots of a released Click source task: concurrent base and completed views produce 39/40 and 40/40 tests, and switch/rollback changes both the selected object and the task outcome. RQ2 compares the same policies with feature-equivalent FUSE: the matched Agent lifecycle is 11.32× [11.24, 11.64], and cache-hot FxMark `SELECT` reaches 1.052–1.088× cached-FUSE throughput across MRPL, MRPM, and MRPH. On corrected FxMark directory enumeration, `SELECT` reaches 2.20–3.66× FUSE throughput in five of six cells; at four shared-directory workers, the ratio is 1.018 [0.907, 1.135], exposing the shared-directory contention boundary. RQ3 executes a matched Wrapfs-derived implementation; both mechanisms pass 37/37 workspace oracles and all 21 fault cells in each of three boots, while runtime tracing exposes the broader stackable filesystem-method boundary.

The paper's primary contribution is the design and Linux implementation of NAMEI_EXT as one systems boundary: a `sched_ext`-style VFS extension point that places bounded policy at lookup and directory enumeration while preserving VFS and lower-filesystem ownership. The paper makes three concrete contributions:

- the boundary design and policy contract for lookup and readdir (Section 4);
- a modified-Linux prototype with one eBPF decision function, the real `cgroup/namei_ext` attachment path, and kernel validation for bounded actions (Section 5); and
- an evaluation that holds workload correctness fixed while comparing NAMEI_EXT with feature-equivalent FUSE and recording which responsibilities would move into custom or stackable filesystems (Section 6).

2 Background

2.1 VFS Name Resolution

The Linux VFS resolves a pathname by walking directory components, consulting directory entries and inodes, and then dispatching ordinary file operations to the selected filesystem [43, 44]. Lookup selects the object on which later file semantics operate, while the VFS and lower filesystem retain permission checks, reads and writes, page-cache behavior, persistence, and consistency. The lookup/operation separation is the boundary the paper uses for path-view policy.

2.2 Programmable Kernel Policy

eBPF lets the kernel run verified, bounded programs at explicit attachment points [34, 35]. The `sched_ext` scheduler class is a useful precedent because it lets BPF choose scheduling policy while the kernel retains scheduler machinery [36]. NAMEI_EXT applies the same policy-versus-ownership separation to VFS name resolution: BPF supplies a bounded path-view decision, and the kernel retains the VFS code that owns dentries, inodes, and object lifetime. The analogy concerns the ownership split rather than the scheduler interface.

2.3 Namespace Construction And Filesystem Services

Linux already offers mechanisms for alternate filesystem views. Bind mounts, OverlayFS, projected volumes, and copied trees construct or materialize a namespace before the application uses it [22, 42, 18]. FUSE and stackable or custom filesystems provide a broader programmable filesystem boundary, often by placing a service or filesystem implementation on the operation path [37, 51, 24]. These mechanisms define the neighboring boundaries for the path-view policy studied in the rest of the paper.

3 Motivation And Workloads

A VFS name-resolution extension targets a narrow workload pattern in which the source system changes which existing object a pathname should select, but the operation does not need a new filesystem object model. The source-system evidence below identifies that pattern and separates it from the broader behavior that should remain with agent runtimes, environment builders, services, FUSE filesystems, or custom filesystems.

3.1 State-Dependent Path Views

Under this definition, a transition qualifies when a system state change alters which existing object a pathname denotes, or whether the object is visible, while ordinary lower-filesystem semantics remain unchanged. The definition is intentionally narrower than "programmable filesystem." It excludes synthetic file contents, custom metadata persistence, distributed directory indexing, data-path mediation, write-conflict resolution, and cross-path transactional snapshots.

Within this definition, the transition is the workload state change, the policy is the eBPF decision logic, and the action is the bounded result that the kernel validates for lookup or directory enumeration.

The distinction matters because the natural implementation boundary changes. If the system needs synthetic contents or custom metadata persistence, a FUSE service, custom filesystem, metadata service, or application/runtime mechanism is the appropriate implementation boundary. If the state change only affects lookup or directory visibility, a VFS name-resolution policy can express the relevant policy. The remaining source-system behavior stays outside name resolution.

3.2 Workload Sources

We use source systems to choose workloads and correctness conditions, not as separate contributions. We call each extracted correctness condition a source oracle. A same-oracle KVM run applies that condition unchanged to NAMEI_EXT and its comparison rows.

The first source family is agent/workspace state. AgentFS is the primary source because its public implementation exposes workspace lifecycle traces from which path-selection and visibility oracles can be extracted [45]. SWE-Factory-Gym supplies a released Click source task whose fail-to-pass test can run

Example	State and operation	NAMEI_EXT action and delegated behavior	Oracle
Agent workspace lifecycle	branch or checkpoint state changes lookup and readdir under a workspace path	select or hide existing lower objects while the source runtime keeps COW, checkpoint, sandbox, writes, and lifecycle orchestration	final tree, lookup/readdir coherence, and preserved lower-filesystem permissions
Application file sharing	application identity and grant state change whether one existing host document is visible	select the granted document or hide it while grant persistence and portal UI remain outside the hook	grant, revoke, cross-application isolation, and unchanged lower object
Build action sandboxing	action identity and declared-input set choose which existing input tree one logical pathname exposes	select the action's declared object or hide an undeclared name while Bazel keeps process and build semantics	real action success, distinct concurrent outputs, and undeclared-input failure

表 1: Representative state-dependent path-view examples. The NAMEI_EXT evidence is the lookup/readdir action, and non-name-resolution behavior remains with the source runtime or lower filesystem.

through those workspace views [11, 32]. BranchFS, Sandlock, and YoloFS provide neighboring branch, sandbox, staging, and stackable-filesystem context rather than separate evaluation workloads [25, 26, 50].

The traditional source families are application file sharing, build action sandboxing, checkpoint/restore, HPC staging, service rotation, and toolchain environments. The application-sharing workload follows the XDG Documents portal grant/revoke behavior [47]. The build-action workload follows Bazel's action-specific input views [3]. The toolchain workload follows per-environment software selection and switch/rollback behavior represented by profile systems such as Nix [27]. Kubernetes AtomicWriter supplies the completed already-materialized payload-view sequence [18]. DMTCP path virtualization supplies an additional source-defined lifecycle shape but remains a portfolio case until its exact KVM oracle runs [2]. Full projected-volume materialization and service reload also remain outside the completed Kubernetes subset.

A historical ccache matrix supplies supporting compile-time evidence, but is not a headline source workload: ccache already implements cache lookup and validation in userspace. Source systems are admitted only when the behavior under study is the application's path view, not an application-level cache algorithm re-expressed in BPF.

Only source behavior tied to a concrete same-oracle KVM run becomes final evaluation evidence. Source inspection and paper-derived behavior motivate workload shape and boundary comparisons.

Three design constraints follow from the source systems. First, each workload carries its own state, such as a branch identifier, checkpoint, cache epoch, build generation, service epoch, or secret epoch. Policy therefore must run at the affected lookup or directory-enumeration operation and remain scoped per workload.

Second, the path-view portion of each source system is narrower than the full system. Many rows also include runtime orchestration, service reload, synthetic contents, or storage semantics that remain separate responsibilities. The narrower path-view scope implies that the kernel and lower filesystem must keep data, permission, and persistence ownership.

Third, directory visibility and lookup selection appear together in the source oracles. Lookup and readdir therefore need one coherent decision contract.

Together, these source-system observations set the paper's comparison rule: (1) source behavior supplies the oracle, (2) feature-equivalent FUSE supplies the direct programmable-policy cost comparison, and (3)

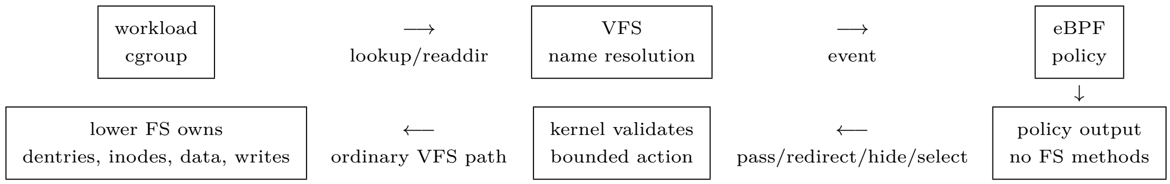

图 1: NAMEI_EXT keeps policy at VFS name resolution. The policy chooses bounded path-view actions, while the kernel and lower filesystem continue to own filesystem objects and data semantics.

custom or stackable filesystems define the broader safety and ownership boundary. Materialized namespace mechanisms remain related-work and background comparisons rather than the central experiment. The same rule drives the design requirements in Section 4 and the RQ-organized evaluation in Section 6.

3.3 Selected Workloads

The evaluated RQ1 set contains five non-overlapping workflows. The formal Agent workspace case exercises branch-visible staged/base selection, hides, readdir coherence, and lower-tree mutations. It also runs a released Click source task through concurrent base/completed views, switch, rollback, and withdrawal. Formal application-sharing and build-action runs exercise per-application grant/revoke and two concurrent real Bazel actions with distinct declared-input roots and an undeclared-input failure. The formal Kubernetes case follows the official `AtomicWriter` payload-view sequence through update, no-op, and rollback under one stable logical root. The formal toolchain case selects two existing CPython environments, switches one process-group view, and rolls it back. These four supporting workflows broaden the source-system evidence without becoming separate headline claims.

Full service validation and reload, checkpoint/restore, and HPC file staging remain portfolio extensions rather than completed experiments. A workload becomes evidence only after a reviewed KVM same-oracle run. In every completed case, policy runs at lookup and directory enumeration; non-name-resolution behavior stays with the source system or lower filesystem.

4 Design

4.1 System Overview

NAMEI_EXT places a programmable decision at the point where the VFS chooses the visible object for a pathname or directory entry. Four invariants organize the design: (1) policy runs only at lookup and directory enumeration, (2) policy returns bounded path-view actions over existing lower objects, (3) the kernel validates every action before ordinary VFS execution resumes, and (4) the lower filesystem retains permissions, file operations, data, writes, page cache, persistence, and consistency.

Figure 1 shows the resulting flow. An application in a workload scope performs an ordinary lookup, open, stat, exec, or readdir. The VFS reaches the NAMEI_EXT decision point before committing to the visible object for the current component or directory entry. The eBPF policy returns a bounded path-view action, and ordinary VFS and lower-filesystem logic resumes after kernel validation.

Goal	Design choice	RQ
G1 Expressiveness	invoke one workload-scoped decision contract at lookup and directory-enumeration time	RQ1
G2 Lower-filesystem ownership	return only bounded actions over existing lower objects while keeping data, writes, permissions, page cache, and persistence below the hook	RQ1, RQ3
G3 Placement cost	run bounded policy at lookup/readdir instead of routing the same decision through a filesystem service	RQ2
G4 Fail-closed safety	use verified eBPF outputs plus kernel validation so malformed or unsupported decisions fail visibly	RQ3

表 2: Design-goal mapping. If an oracle needs synthetic contents, custom metadata, or data-path mediation, the effect belongs outside NAMEI_EXT's boundary.

4.2 Name-Resolution Boundary Goals

Four goals follow from keeping policy at name resolution while leaving filesystem ownership below the hook: expressiveness, lower-filesystem ownership, placement cost, and fail-closed safety. Like `sched_ext`, NAMEI_EXT lets BPF choose policy while the kernel keeps the VFS mechanisms that own objects and execute filesystem semantics. The VFS and lower filesystem retain path walking, dentries, inodes, permissions, file operations, page cache, writes, persistence, and consistency.

The design defines a narrow kernel extension for name-resolution policy while leaving full filesystem expressiveness to FUSE, stackable, or custom filesystems. Table 2 maps the four design goals to the RQs.

4.3 Policy Model

The workloads require policy to see lookup and directory-enumeration events, carry workload state, return only bounded path-view actions, and let the kernel validate the result.

The policy input contains operation context, the current component, parent context, and policy state. The policy state is workload-defined but not a filesystem object model. For the representative workloads, policy state is a branch or checkpoint identifier in an agent workspace, a cache or validation epoch in an environment run, or a configuration/secret epoch in a service.

The output passes the lower lookup, redirects selection to a replacement component, hides a name from lookup and directory enumeration, or selects a registered lower object. Access-control denial remains outside the model because NAMEI_EXT focuses on selecting or hiding existing objects during name resolution. Directory enumeration uses the same decision function so that aliases returned by readdir match the objects selected by lookup.

The kernel checks the decision, preserves normal permission and lower-filesystem checks for the selected object, maintains ordinary VFS path resolution invariants, and fails invalid decisions visibly instead of falling back to a weaker policy.

4.4 Filesystem Ownership Boundary

NAMEI_EXT excludes filesystem ownership by leaving inode operations, file operations, dentry and inode allocation, synthetic contents, custom metadata, read/write mediation, distributed indexes, and cross-path transactions outside the hook.

The name-resolution boundary helps only when the source transition can be expressed as lookup selection or directory visibility. When the oracle requires data-path mediation, synthetic content, custom metadata, or application-level orchestration, the transition remains motivation or related work rather than NAMEI_EXT evidence.

5 Implementation

The Linux prototype turns the policy contract into a real `cgroup/namei_ext` attachment path. Policies are eBPF programs under `bpf/policies/*.bpf.c`. The implementation consists of the VFS call sites, the BPF ABI, user-space policy attachment and map setup, and kernel-side validation for the supported action set.

5.1 Kernel Attachment

The lookup path calls the policy before the VFS commits to the lower object for the current component or final-component create/open path. Directory enumeration calls the same decision function for entries that may be exposed under an alias. Both call sites preserve the normal lower-filesystem path after the policy returns a validated decision, so the VFS still applies ordinary permission checks and file operations to the selected object.

5.2 BPF ABI

The kernel-facing ABI exposes one decision function. The context includes an event type, operation flags, the current component name, parent identity, identifiers used by the policy, and bounded output fields for the selected path-view action. The current action set is `PASS`, `REDIRECT`, `HIDE`, and `SELECT_TARGET`. `REDIRECT` carries a bounded replacement component in `redirect_name`. `SELECT_TARGET` carries an opaque `target_id` that indexes a kernel-registered lower `struct path`. The policy cannot return arbitrary unbounded path strings or perform recursive path walks.

The kernel validates action type, bounded names, target identity, and selection scope before continuing. Redirect decisions are validated for length and same-parent scope. Hide returns absence on lookup and suppresses the lower entry during directory enumeration.

5.3 Prototype Coverage

The prototype implements pass-through, validated same-parent replacement-component redirect, hide, and `SELECT_TARGET` over kernel-registered lower paths. Registered lower-path selection supports intermediate components and final selected lower objects while leaving data, permissions, and file operations to the lower filesystem.

The hook does not synthesize parent-directory aliases or create missing selected targets. Creation, cache population, and lifecycle orchestration remain responsibilities of the source runtime or lower filesystem.

5.4 Replacement-Component Validation And Permission Preservation

Workload setup configures the policy state that chooses replacement components. The prototype implements the redirect subset by copying a bounded `redirect_name` from the BPF output, validating its length, action, and same-parent scope, and then resuming the normal VFS path. The replacement component is not an arbitrary path string and does not trigger a recursive path walk. After validation, the normal VFS path continues, including permission checks and lower-filesystem file operations. Same-parent validation keeps NAMEI_EXT from becoming arbitrary path rewriting or a filesystem service.

The implemented hide action uses the same decision point but does not allocate or synthesize filesystem objects. Lookup observes absence, and directory enumeration suppresses the lower entry for the active policy state.

`SELECT_TARGET` adds kernel-owned object identity and lifetime validation before a policy can select a registered lower path outside the visible parent.

5.5 Failure Handling

Unsupported actions, verifier failures, malformed decisions, syscall failures, and invalid lower selections are hard failures in the owning Make target. The implementation does not silently fall back to a weaker policy when validation fails. This fail-closed behavior is part of the safety boundary evaluated in RQ3.

Phase 1 validation runs in KVM with the modified kernel and the real `cgroup/namei_ext` attach path. Host-only checks, object inspection, and bpftool-only smoke tests are diagnostics rather than validation for the paper's mechanism claims.

These components give the evaluation a single validation path in which Make targets boot the modified kernel, attach policies through `cgroup/namei_ext`, and preserve raw validation and experiment artifacts.

6 Evaluation

6.1 Research Questions

The evaluation is organized around three research questions. Each answer is conditioned on a source oracle: the same workload behavior must be used for NAMEI_EXT, the feature-equivalent FUSE row, and the custom/stackable responsibility comparison. One-off mechanism checks, source surveys, and weak baselines do not answer an RQ by themselves.

RQ1 Can a narrow VFS name-resolution extension express real state-dependent path-view policies without taking over filesystem semantics?

RQ2 What is the cost of putting programmable policy on the VFS name-resolution path compared with a feature-equivalent FUSE policy implementation?

RQ3 Does NAMEI_EXT provide a narrower verifier-bounded, fail-closed ownership boundary than building a custom or stackable filesystem when the needed policy is only name resolution?

6.2 Experimental Setup

All NAMEI_EXT mechanism runs execute in KVM [39] with the modified kernel and the real `cgroup/namei_ext` attach path. Make targets own the run, attach the policy, collect raw artifacts, and fail on missing capability, verifier failure, syscall failure, or oracle failure. Each final row records the kernel commit, image identity, policy object, workload inputs, stdout/stderr, dmesg, per-operation lookup/readdir events, and comparison-specific setup observations. Operation-weighted metrics weight each policy action by its observed frequency in the workload trace.

Each final reported run records the kernel configuration and image identity, filesystem configuration, run count, warmup rule, confidence-interval method, cache protocol, compared implementation, source commit, and raw observations.

Correctness gates every number. A FUSE row is used for RQ2 only after it implements the same policy over the same source oracle. No-hook and lower-filesystem rows calibrate fixed hook and measurement cost. They are not competing baselines. RQ3 executes a matched Wrapfs-derived implementation for the Agent workspace oracle and compares responsibility and containment rather than throughput.

6.3 RQ1: Expressiveness And Sufficiency

RQ1 asks whether the VFS boundary can express path-view transitions extracted from source oracles while preserving lower-filesystem semantics. A reportable row must pass the source oracle through the real KVM attach path and show that NAMEI_EXT changed only lookup or directory visibility, while runtime orchestration, storage state, synthetic contents, writes, and data-path behavior remain with the source system or lower filesystem.

Workload	Source oracle	NAMEI_EXT evidence	Result evidence
Application file sharing (supporting) [47]	The XDG Documents portal grant state selects one existing host document for one application and hides it from another.	NAMEI_EXT selects or hides the document by application and grant state.	Three fresh KVM boots passed 15/15 states: 3 granted views, 12 hidden views, exact logical/lower object identity, and unchanged lower and unrelated objects.
Agent workspaces [45, 11, 32]	AgentFS-derived branch state selects staged/base objects and hides whiteouts; a released Click task distinguishes base from completed source.	NAMEI_EXT selects or hides existing lower objects; the lower filesystem retains permissions, writes, metadata, and file data.	The lifecycle matrix recorded 1,170 pass-bearing records with zero failed oracles plus six metadata records in each of three runs. Three fresh source-task boots passed 12/12 policy-backed task states and 6/6 physical source controls: concurrent base/completed views produced 39/40 and 40/40 tests, followed by switch, rollback, and withdrawal.
Build action sandboxing (supporting) [3]	Two concurrent Bazel actions use one logical input path but different declared roots; an undeclared existing input must be absent.	NAMEI_EXT selects each action's declared object and hides the undeclared name.	Three fresh KVM boots completed all six Bazel 6.5.0 actions, matched six logical/lower input objects, hid undeclared inputs, and preserved 12 lower objects.
Toolchain environments (supporting) [27]	Two installed environments expose distinct interpreter and package trees; one process-group view switches and rolls back.	NAMEI_EXT selects the existing environment root without changing the logical executable path.	Three fresh KVM boots passed 18/18 physical/logical states, 24/24 Python probes, concurrent 3.10/3.12 views, switch, rollback, and lower-object controls.
Kubernetes ConfigMap publication (supporting) [18, 19]	Official `AtomicWriter` publication exposes V0, V1, repeated V1, and rollback V0 payloads under one stable root.	NAMEI_EXT selects or hides already-materialized leaf objects while VFS retains modes, ownership, open descriptors, and lower objects.	Three fresh KVM boots passed 12/12 source states, 12/12 NAMEI_EXT states, 6/6 direct controls, 24/24 stable-root descriptor checks, 12/12 old-descriptor checks, and 36/36 lower-object checks.

表 3: RQ1 answer table. A row becomes evidence only after the source-oracle run passes through the real KVM attach path and lower-filesystem semantics are preserved.

The required RQ1 evidence is a reviewed KVM pass/fail matrix that preserves lookup/readdir coherence and lower-filesystem behavior for each admitted source oracle. A workload is outside NAMEI_EXT's boundary if the oracle needs synthetic contents, data-path mediation, write-conflict handling, or custom metadata ownership in the common path.

6.4 RQ2: Cost Versus FUSE

RQ2 isolates the placement cost of policy by comparing NAMEI_EXT with a feature-equivalent FUSE policy service under the same oracle. Non-name-resolution lifecycle behavior is delegated identically in both rows. RQ2 does not compare different runtimes. Instead, it compares placing the same lookup/readdir decision inside VFS name resolution with placing it in a FUSE request path.

Workload	Feature-equivalent FUSE comparison	Metrics	Result evidence
Agent workspace lifecycle	Same 48-oracle lifecycle in NAMEI_EXT and a feature-equivalent FUSE policy service.	Ten paired blocks, 20 KVM boots, 10,000 samples per mechanism.	960/960 oracles passed. Lifecycle p50 was 5.51 μs versus 62.64 μs; paired FUSE/NAMEI_EXT ratio was 11.32× [11.24, 11.64].
FxMark cache-hot path walks	Stock, patched-unattached, attached `PASS`, same-filesystem `SELECT`, and multithreaded cached FUSE.	50 fresh KVM boots, 450 cells over MRPL, MRPM, and MRPH at one/two/four workers.	450/450 cells passed. `SELECT`/FUSE was 1.052–1.088×, with every paired 95% CI above one; complete `SELECT` retained 0.895–0.931 of unattached throughput.
Corrected FxMark directory enumeration	Same five conditions over private-directory MRDL and shared-directory MRDM.	Ten paired blocks, 50 fresh KVM boots, and 300 cells at one/two/four workers.	300/300 cells passed. `SELECT`/FUSE was 2.20–3.66× with CIs above one in five cells; MRDM/4 was 1.018 [0.907, 1.135].
Unused fast path	Stock versus patched-unattached cache-hot MRPL in 30 paired blocks.	60 fresh KVM boots and 180 cells.	Unattached/stock was 1.0009 [0.9921, 1.0036], 1.0083 [0.9950, 1.0179], and 1.0007 [0.9918, 1.0139] at one/two/four workers.

表 4: RQ2 evidence table. FUSE numbers are interpreted only after the FUSE row implements the same source policy and passes the same source oracle.

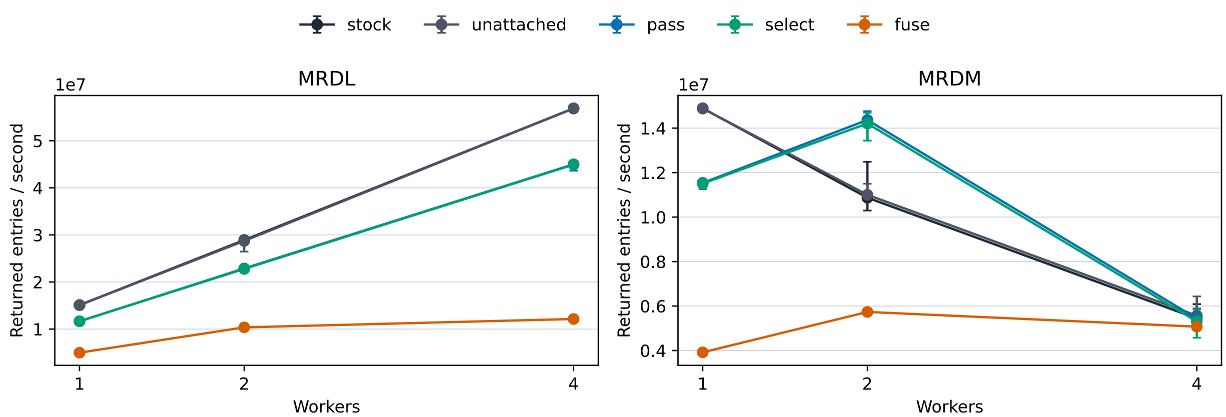

图 2: Corrected FxMark directory-enumeration throughput. Points are medians over ten fresh-boot blocks; error bars are 95% bootstrap intervals. MRDL gives each worker a private directory; MRDM makes all workers enumerate one shared directory.

The Agent result is a mechanism-level lifecycle comparison, not an end-to-end agent-task speedup. The FxMark FUSE view enabled metadata and kernel caching and issued only 0–19 measured requests per cell; its result does not depend on one daemon round trip per lookup. Attached `PASS` retained 0.901–0.934 of unattached throughput, while `SELECT`/`PASS` was 0.981–0.999. The separate unused-path run supports only cache-hot MRPL on this host; cold caches, other operations, tail latency, and other machines remain open.

Figure 2 broadens RQ2 to the event where the BPF policy runs for every returned directory entry. For private directories, `SELECT`/FUSE is 2.200–3.663×, and `SELECT` scales 3.844× from one to four workers versus 2.411× for FUSE. Shared-directory enumeration is 2.909× and 2.450× at one and two workers. At four workers, stock, `SELECT`, and FUSE converge near five million entries/s; the `SELECT`/FUSE interval crosses one. The frozen overall verdict is therefore mixed: five cells support the throughput advantage, while shared-directory contention masks it in the sixth.

6.5 RQ3: Ownership And Containment Boundary

RQ3 compares ownership boundaries for the same oracle-relevant behavior. The question is not whether custom or stackable filesystems are expressive, because they are. The question is whether the workload needs that broader boundary when the policy effect is only name-resolution selection or visibility.

Evidence	NAMEI_EXT	Wrapfs-derived	Runtime/fault observation	Result
Agent workspace, same 37-row oracle	BPF decides lookup/readdir; ordinary operations use the selected ext4 object.	A port of official Wrapfs registers 34 unique VFS slots across six compiled sources.	Thirteen Wrapfs method classes executed. After NAMEI_EXT detach, an open descriptor completed ordinary lower-file operations without re-entering BPF.	37/37 oracles passed for both mechanisms in each of three KVM boots.
Invalid policy/output matrix	Verifier rejects invalid programs; kernel returns declared errors for malformed or unsupported decisions.	Validation and failures remain kernel-module filesystem responsibilities.	Two verifier cases produced exact rejection logs; 19 independent runtime faults each preserved exact lower-object evidence.	21/21 cells passed in each of three boots.

表 5: RQ3 boundary table. The comparison is responsibility and containment for the same oracle-relevant behavior, not a claim that custom filesystems are less expressive.

For this existing-object workspace view, RQ3 is supported: NAMEI_EXT confines workload policy execution to lookup and directory iteration, while the matched stackable implementation owns and executes superblock, lookup, directory, inode, and file methods. This is a workload-specific ownership and containment result. It is not proof of complete-system security, nor a claim that custom filesystems are unnecessary when an oracle requires synthetic contents, copy-up, conflict resolution, distributed metadata, or data-path mediation.

6.6 Evaluation Scope

The completed evidence includes five reviewed formal RQ1 workflows, an Agent lifecycle RQ2 comparison, cache-hot FxMark active and unused-path results, a formal corrected-FxMark directory-enumeration matrix, and one matched Agent-workspace RQ3 boundary experiment. A historical ccache compile matrix is supporting macro evidence: its hot-cache run observed FUSE/NAMEI_EXT 2.18× and native/NAMEI_EXT 0.945×, but it lacks independent-run uncertainty and release-scale miss/stale/corrupt cells. It is not a headline workload because ccache already implements cache lookup and validation in userspace.

Full service validation/reload, checkpoint/restore, and HPC staging remain portfolio extensions rather than completed evidence. The Kubernetes result covers already-materialized payload selection and visibility, not ConfigMap retrieval, symlink/inotify behavior, or application reload. Cache-cold lookup, broader metadata operations, tail latency, and a second source-derived RQ3 row remain open. Materialized namespaces, eBPF LSM, and native source mechanisms remain related work unless a fixed source oracle makes one a direct comparison.

7 Discussion

7.1 Implications

The broader lesson is that narrow programmable hooks help when applications vary policy but the kernel should retain mechanism ownership. NAMEI_EXT applies that lesson to VFS name resolution rather than to a new filesystem. For NAMEI_EXT, the retained mechanism is VFS path walking and lower-filesystem object semantics. The NAMEI_EXT boundary is useful only when the correctness condition is pathname-to-object selection or directory visibility over existing objects.

FUSE, stackable filesystems, and custom filesystems remain the appropriate implementation boundary when a workload needs filesystem-service features such as synthetic contents, persistent custom metadata, data-path mediation, write-conflict resolution, distributed metadata, or cross-path transactions. NAMEI_EXT is intended to avoid that boundary only for the subset of policies whose effect is visible during lookup or directory enumeration.

7.2 Extension Path

The same boundary also defines future extensions. Synthetic aliases require additional kernel support, while synthetic contents, custom metadata persistence, distributed indexing, and cross-path transactions remain filesystem or runtime responsibilities. NAMEI_EXT is intended only for workloads whose correctness oracle is path selection or visibility over existing lower objects.

8 Related Work

Related work falls along a spectrum of filesystem ownership. Namespace mechanisms construct views before lookup, and access-control hooks mediate or observe operations near the VFS. NAMEI_EXT puts bounded object-selection policy in VFS name resolution, while FUSE, stackable filesystems, custom filesystems, and metadata services require broader filesystem-service or object-model ownership.

8.1 FUSE Policy Services

FUSE [37], FUSE passthrough [38], ExtFUSE [4], and FUSE-BPF [31] demonstrate the value of placing filesystem policy in, or accelerating, a filesystem-service boundary. RFUSE, CoFS, and DFUSE further show that this boundary can be optimized or specialized with scalable kernel-userspace communication, fixed container-image lookup structures, and distributed-cache consistency support [9, 46, 20]. These systems make FUSE the RQ2 baseline family, not a strawman: a fair comparison must implement the same path-view policy under the same source oracle and justify the FUSE caching/passthrough configuration. NAMEI_EXT instead keeps policy needed by the source oracle at VFS name resolution when the workload only needs path selection or visibility over existing objects.

8.2 Custom, Stackable, And Metadata-Service Filesystems

Wrapfs [51], Bento [24], and custom filesystems demonstrate broader filesystem-extension boundaries. They can implement richer behavior and, in Bento's case, reduce the risk of in-kernel filesystem development. They are the RQ3 ownership comparison point because the developer still owns filesystem methods, storage semantics, or framework-specific filesystem behavior. These mechanisms remain the appropriate implementation boundary when the workload needs custom file operations, synthetic contents, persistence, or richer semantics.

DeltaFS, IndexFS, and TableFS show that full metadata services and specialized filesystems are valuable when the problem is metadata distribution, indexing, or storage layout [53, 30, 29]. Recent systems reinforce the same boundary: Oxbow coordinates kernel, userspace, and device components; DeLFS builds a decentralized kernel filesystem; FalconFS moves path resolution and metadata indexing into a distributed filesystem service; SwitchFS and MesaFS target distributed metadata updates; and SYSSPEC/SpecFS generates a complete filesystem from specifications [16, 1, 49, 48, 14, 23]. They are not the main workload baselines for NAMEI_EXT, but they clarify the boundary because such workloads require a metadata service or full filesystem when the oracle needs distributed metadata, data-path ownership, persistence, or a new filesystem object model.

8.3 Namespace Construction

Bind mounts, projected volumes, symlink forests, copy trees, and OverlayFS are practical ways to present alternate views [18, 42]. They preserve ordinary kernel filesystem behavior, but express policy by constructing or updating namespace state. NAMEI_EXT asks when the policy can stay at lookup and directory-enumeration time instead of materializing a view before the transition completes.

8.4 Access Control And Observation Hooks

Landlock, BPF LSM programs, and fanotify show that Linux already exposes specialized hooks around filesystem access and events [40, 41, 21]. Among neighboring mechanisms, these hooks sit between materialized namespace construction and NAMEI_EXT. They can run verified policy in the kernel, but their natural role is to deny, mediate, or observe operations. NAMEI_EXT selects which existing object a name denotes during resolution and then returns control to the ordinary VFS and lower filesystem. Access-control denial remains outside the current action set because the primary claim is name-resolution selection and visibility. USEC is a recent access-control neighbor with an LSM-compatible mandatory-access-control design [15]; XRP, vBPF, PeeR, KRAKENGUARD, Xkernel, and bpftime/EIM are adjacent programmability work on storage-function hooks, eBPF virtualization, eBPF scheduling, eBPF isolation, kernel tunability, and userspace extension safety [55, 52, 5, 28, 8, 54]. They help set safety and overhead expectations for programmable kernel extensions, but they do not provide a VFS name-resolution object-selection boundary.

8.5 Source Workload Systems

Agent filesystems are closest to our workloads, but they are workload sources rather than direct cost baselines [45, 25, 26, 50]. They show that agent workspaces need branch, sandbox, staging, and snapshot-like views, while their implementations also own behavior beyond NAMEI_EXT's boundary. For agent workloads, custom or stackable filesystem ownership provides the RQ3 responsibility comparison, while FUSE remains the RQ2 cost comparison. NAMEI_EXT applies when the same source oracle can be satisfied by changing path visibility or object selection while leaving the rest of that ownership with the source system or lower filesystem.

Environment and cache systems provide a second workload source, not a filesystem mechanism baseline. SWE-Factory-Gym also supplies the released source-task oracle used in the Agent workspace case. Along with MEnvData-SWE and SWE-rebench V2, it provides build/test oracles for environment state and cache reuse policies [11, 32, 13, 33]. CoFS and large-scale LLM data-pipeline work add contemporary evidence that build, container, and data-cache paths matter to real systems [46, 7]. Agentic systems such as Murakkab and

SREGym show that modern agents run through multi-step tool, workflow, and live-environment oracles [6, 10]. These sources motivate workload state and oracle selection. Their runtime orchestration, environment synthesis, projected-volume materialization, and reload behavior remain outside NAMEI_EXT's mechanism claim.

9 Conclusion

NAMEI_EXT frames state-dependent path views as a systems problem in which pathname policy is programmable while filesystem ownership remains with the VFS and lower filesystem. The abstraction occupies the space between namespace materialization and full programmable filesystems, following the same policy-versus-ownership split used by `sched_ext`. The design and prototype show that lookup-time policy can be exposed as one bounded eBPF decision while VFS and lower filesystems retain object ownership. The evaluation therefore centers on three lookup/readdir claims: expressiveness for source-derived path-view policies, placement cost versus feature-equivalent FUSE, and ownership and containment versus custom or stackable filesystems.

参考文献

[1] Taehwan Ahn, Chanhyeong Yu, Sangjin Lee, and Yongseok Son. DeLFS: A decentralized Log-Structured file system for manycores. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 793–807, Seattle, WA, July 2026. USENIX Association.

[2] Jason Ansel, Kapil Arya, and Gene Cooperman. Transparent checkpointing of the whole software stack: the dmtcp approach. In *Proceedings of the 2016 Workshop on Fault Tolerance for HPC at Extreme Scale*, 2016.

[3] Bazel. Sandboxing. `https://bazel.build/docs/sandboxing`. Accessed 2026-06-14.

[4] Ashish Bijlani and Umakishore Ramachandran. Extension framework for file systems in user space. In *USENIX ATC*, 2019.

[5] Jeremy Carin, Ben Holmes, Weiyang Wang, Ankit Bhardwaj, and Manya Ghobadi. PeeR: First-Class scheduling for Latency-Critical eBPF applications. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 2465–2482, Seattle, WA, July 2026. USENIX Association.

[6] Gohar Irfan Chaudhry, Esha Choukse, Haoran Qiu, Inigo Goiri, Rodrigo Fonseca, Adam Belay, and Ricardo Bianchini. Murakkab: Resource-Efficient agentic workflow orchestration in cloud platforms. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 567–587, Seattle, WA, July 2026. USENIX Association.

[7] Luofan Chen, Chenhan Wang, Weidong Zhang, Jinxin Chi, Hequan Zhang, Zanbo Wang, Chenyuan Wang, Lishu Luo, Sijin Wu, Junqi Hu, Jun Wang, Cheng Chen, Lixin Huang, Liyang Zhao, Yong Tian, Jun Guo, Youhui Bai, Wencong Xiao, Kang Chen, and Cheng Li. Teaching the old dog new tricks: Building efficient data pipelines for Large-Scale LLM Pre-Training (operational systems). In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 349–365, Seattle, WA, July 2026. USENIX Association.

[8] Zhongjie Chen, Wentao Zhang, Yulong Tang, Ran Shu, Fengyuan Ren, Tianyin Xu, and Jing Liu. Xkernel: Principled performance tunability of operating system kernels. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 547–565, Seattle, WA, July 2026. USENIX Association.

[9] Kyu-Jin Cho, Jaewon Choi, Hyungjoon Kwon, and Jin-Soo Kim. RFUSE: Modernizing userspace filesystem framework through scalable Kernel-Userspace communication. In *22nd USENIX Conference on File and Storage Technologies (FAST 24)*, pages 141–157, Santa Clara, CA, February 2024. USENIX Association.

[10] Jackson Clark, Yiming Su, Saad Mohammad Rafid Pial, Yifang Tian, Lily Gniedziejko, Hans-Arno Jacobsen, Yinfang Chen, and Tianyin Xu. SREGym: A live benchmark for AI SRE agents with High-Fidelity failure scenarios. arXiv:2605.07161, 2026.

[11] DeepSoftwareAnalytics. Swe-factory. `https://github.com/DeepSoftwareAnalytics/swe-factory`, 2025. Accessed 2026-07-10.

[12] Docker Docs. What is docker? `https://docs.docker.com/get-started/docker-overview/`, 2026. Accessed 2026-07-13.

[13] ERNIE Research. Menvagent and menvdata-swe. `https://github.com/ernie-research/MEnvAgent`, 2026. Accessed 2026-07-10.

[14] Hao Guo, Jiwu Shu, and Youyou Lu. MesaFS: An I/O-Efficient metadata service for distributed file systems. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys 26)*, 2026.

[15] Yu Jiang, Wenhuan Liu, Fuchen Ma, Yuheng Shen, Yuanliang Chen, Lei Zhang, He Li, Quan Zhang, and Chijin Zhou. USEC: A User-Requirement-Driven mandatory access control framework for operating systems (operational systems). In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 191–206, Seattle, WA, July 2026. USENIX Association.

[16] Jongyul Kim, Jaehwan Lee, Inhoe Koo, Peizhe Liu, Jiyuan Zhang, Junho Ahn, Tianyin Xu, and Youngjin Kwon. Oxbow: A coordinated architecture for Multi-Component file systems. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 1425–1442, Seattle, WA, July 2026. USENIX Association.

[17] Kubernetes. Secrets. `https://kubernetes.io/docs/concepts/configuration/secret/`. Accessed 2026-06-14.

[18] Kubernetes. Projected volumes. `https://kubernetes.io/docs/concepts/storage/projected-volumes/`, 2026. Accessed 2026-06-14.

[19] Kubernetes documentation. Configure a pod to use a configmap. `https://kubernetes.io/docs/tasks/configure-pod-container/configure-pod-configmap/`, 2026. Accessed 2026-07-13.

[20] Haoyu Li, Jingkai Fu, Qing Li, Windsor Hsu, and Asaf Cidon. DFUSE: Strongly consistent Write-Back kernel caching for distributed userspace file systems. arXiv:2503.18191, 2025.

[21] Linux man-pages. fanotify(7). `https://man7.org/linux/man-pages/man7/fanotify.7.html`, 2026. Accessed 2026-06-14.

[22] Linux man-pages. mount(2) linux manual page. `https://man7.org/linux/man-pages/man2/mount.2.html`, 2026. Accessed 2026-07-13.

[23] Qingyuan Liu, Mo Zou, Hengbin Zhang, Dong Du, Yubin Xia, and Haibo Chen. Sharpen the spec, cut the code: A case for generative file system with SYSSPEC. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*, pages 291–311, Santa Clara, CA, February 2026. USENIX Association.

[24] Samantha Miller, Kaiyuan Zhang, Mengqi Chen, Ryan Jennings, Ang Chen, Danyang Zhuo, and Thomas Anderson. High velocity kernel file systems with bento. In *FAST*, 2021.

[25] MultiKernels. Branchfs. `https://github.com/multikernel/branchfs`, 2026. Accessed 2026-07-10.

[26] MultiKernels. Sandlock. `https://github.com/multikernel/sandlock`, 2026. Accessed 2026-07-10.

[27] Nix. Nix store. `https://nix.dev/manual/nix/2.34/store/`. Accessed 2026-06-14.

[28] Jainil Patel, Lucas Graeff Buhl-Nielsen, Adrien Ghosn, and Marios Kogias. KRAKENGUARD: Towards Fine-Grained eBPF isolation. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 2685–2704, Renton, WA, May 2026. USENIX Association.

[29] Kai Ren and Garth Gibson. Tablefs: Enhancing metadata efficiency in the local file system. In *USENIX ATC*, 2013.

[30] Kai Ren, Qing Zheng, Swapnil Patil, and Garth Gibson. Indexfs: Scaling file system metadata performance with stateless caching and bulk insertion. In *SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2014.

[31] Daniel Rosen. FUSE BPF: A stacked filesystem extension for fuse. `https://patchew.org/linux/20230418014037.2412394-1-drosen%40google.com/`, 2023. Accessed 2026-07-14.

[32] SWE-Factory. SWE-Factory-Gym dataset. `https://huggingface.co/datasets/SWE-Factory/SWE-Factory-Gym`, 2026. Dataset row `pallets__click-2622`; accessed 2026-07-29.

[33] SWE-rebench. Swe-rebench v2. `https://github.com/SWE-rebench/SWE-rebench-V2`, 2026. Accessed 2026-07-10.

[34] The Linux Kernel documentation. Bpf documentation. `https://docs.kernel.org/bpf/`, 2026. Accessed 2026-07-13.

[35] The Linux Kernel documentation. ebpf verifier. `https://docs.kernel.org/bpf/verifier.html`, 2026. Accessed 2026-07-13.

[36] The Linux Kernel documentation. Extensible scheduler class. `https://docs.kernel.org/scheduler/sched-ext.html`, 2026. Accessed 2026-06-14.

[37] The Linux Kernel documentation. Fuse overview. `https://docs.kernel.org/filesystems/fuse/fuse.html`, 2026. Accessed 2026-06-14.

[38] The Linux Kernel documentation. Fuse passthrough. `https://docs.kernel.org/next/filesystems/fuse-passthrough.html`, 2026. Accessed 2026-06-14.

[39] The Linux Kernel documentation. Kvm. `https://docs.kernel.org/virt/kvm/index.html`, 2026. Accessed 2026-07-13.

[40] The Linux Kernel documentation. Landlock: unprivileged access control. `https://docs.kernel.org/userspace-api/landlock.html`, 2026. Accessed 2026-06-14.

[41] The Linux Kernel documentation. Lsm bpf programs. `https://docs.kernel.org/bpf/prog_lsm.html`, 2026. Accessed 2026-06-14.

[42] The Linux Kernel documentation. Overlay filesystem. `https://docs.kernel.org/filesystems/overlayfs.html`, 2026. Accessed 2026-06-14.

[43] The Linux Kernel documentation. Overview of the linux virtual file system. `https://docs.kernel.org/filesystems/vfs.html`, 2026. Accessed 2026-07-13.

[44] The Linux Kernel documentation. Pathname lookup. `https://docs.kernel.org/filesystems/path-lookup.html`, 2026. Accessed 2026-07-13.

[45] Turso Database. Agentfs. `https://github.com/tursodatabase/agentfs`, 2026. Accessed 2026-07-10.

[46] Li Wang, Jinxu Du, Yang Yang, Qingbo Wu, Tao Liu, and Haoze Wu. CoFS: A filesystem for fast container startup. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*, pages 415–423, Santa Clara, CA, February 2026. USENIX Association.

[47] XDG Desktop Portal Project. Documents portal. Official API documentation, 2026.

[48] Jingwei Xu, Mingkai Dong, Qiulin Tian, Ziyi Tian, Tong Xin, and Haibo Chen. SwitchFS: Asynchronous metadata updates for distributed filesystems with In-Network coordination. EuroSys 2026; arXiv:2410.08618, 2026.

[49] Jingwei Xu, Junbin Kang, Mingkai Dong, Mingyu Liu, Lu Zhang, Shaohong Guo, Ziyan Qiu, Mingzhen You, Ziyi Tian, Anqi Yu, Tianhong Ding, Xinwei Hu, and Haibo Chen. FalconFS: Distributed file system for Large-Scale deep learning pipeline. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 431–447, Renton, WA, May 2026. USENIX Association.

[50] YoloFS. Yolofs. `https://github.com/YoloFS/YoloFS`, 2026. Accessed 2026-07-10.

[51] Erez Zadok, Ion Badulescu, and Alex Shender. Extending file systems using stackable templates. In *USENIX ATC, FREENIX Track*, 1999.

[52] Jing Zhang, Xiaguannan Song, Dong Du, Yubin Xia, Binyu Zang, and Haibo Chen. Virtualizing eBPF with Late-Binding. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 2127–2148, Seattle, WA, July 2026. USENIX Association.

[53] Qing Zheng, Charles D. Cranor, Gregory R. Ganger, Garth A. Gibson, George Amvrosiadis, Bradley W. Settlemyer, and Gary A. Grider. Deltafs: A scalable no-ground-truth filesystem for massively-parallel computing. In *SC*, 2021.

[54] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. Extending applications safely and efficiently. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 557–574, Boston, MA, July 2025. USENIX Association.

[55] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, and Asaf Cidon. XRP: In-Kernel storage functions with eBPF. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 375–393, Carlsbad, CA, July 2022. USENIX Association.